Assignment #2

Due Date: January 30 2026

Objectives

- Solving recurrence relations
- Reduction
- Divide and Conquer

Submission Instructions

Solve the following problems. Write your solutions on a paper or type them out. Submit your solution to the correspondin dropbox folder. Submit the file as a pdf file.

- Name your file: a1_x.pdf where x is you username as it appears on MyLearningSpace. For example, if I were submitting the assignment, I would name my file: a1_mrudafshani.pdf.
- Write neatly and clearly, or you may lose marks.
- Include the following at the beginning of your submission.

I certify that this assignment is entirely my own work and that all external resources, if used, have been properly acknowledged.

Problem 1

Solve the following recurrence using recurrence tree method. Verify your answer using the general case of the master theorem. [10 points]

$$T(n) = \begin{cases} 2T(n/10) + \sqrt{n} & n > 1 \\ 7 & n \leq 1 \end{cases}$$

Problem 2

Solve the following recurrence by changing variables method. Prove the correction of your solution. [10 points]

$$T(n) = \sqrt{n}T(\frac{n}{2}) + 1$$

Problem 3

An array A of n distinct integers $a_1, a_2, \ldots, a_n$ is known to have the following property: elements follow in descending order up to a certain index p where $1 < p < n$ and then follow in ascending order: $a_1 > a_2 > \cdots > a_{p-1} > a_p < a_{p+1} < \cdots < a_n$. Design an efficient algorithm to find the index of smallest value in this array (i.e., to find p). Note: the worst case running time of your algorithm must be in $o(n)$, note it is little o , so simple scanning from left to right is not going to work. Sublinear-time algorithm is required, with a runtime more efficient than linear time. Detailed pseudocode and runtime analysis is required. [10 points]
Hint: think about binary search.

Problem 4

Given a sequence of numbers $a_1, \ldots, a_n$ find the number of **weighted inversions**. Two indices i and j form a weighted inversion if $i < j$ and $a_i > 2a_j$. You need to design an $O(n \log n)$ divide-and-conquer algorithm to solve this problem. Detailed pseudocode and runtime analysis is required. [10 points]

Problem 5

As discussed in class, we have a faster solution to 2Sum problem in which, given a sorted array, we can find a list of numbers from that array that sum up to a given target in O(n) time rather than $O(n^2)$ time. We saw that 3Sum (where a list of 3 numbers in a sorted array that sum up to a given target) can be found in $O(n^2)$ time rather than $O(n^3)$ time. Please find an algorithm to Solve the 4Sum problem to find numbers in a sorted array that sum up to a target. The solution should be in $O(n^3)$ time. Please explain how we can use the 3Sum solution in it. Please justify your answer. [10 points]